

URIB Canonical Architecture Stack

Specification of the Seven Core Primitives

Document ID	URIB-SPEC-001
Version	1.0.0
Status	Draft for Publication
Date	July 2, 2026
Classification	Public
Working Group	leonlong.research@gmail.com
Contact	LeonLong.research@gmail.com

Abstract

This specification defines the URIB (Universal Resource and Identity Bus) Canonical Architecture Stack and its seven foundational primitives: the Canonical Event Model (CEM), Reversible Mapping (RM), Deterministic Routing (DR), Seed Lifecycle (SL), Settlement Envelope (SE), DID Provenance (DP), and Universal Receipt (UR). These primitives collectively provide a complete, interoperable substrate for deterministic, auditable, and privacy-preserving distributed transactions. Conformance levels URIB-Core, URIB-Standard, and URIB-Full are defined herein.

Table of Contents

Executive Summary	3
1. Introduction	4
2. Primitive 1 — Canonical Event Model (CEM)	5
2.1 Normative Schema	
2.2 Event Classification Taxonomy	
2.3 Serialization and Ordering	
2.4 Immutability and Conformance	
3. Primitive 2 — Reversible Mapping (RM)	7
3.1 Mapping Definition Language	
3.2 Mapping Registry	
3.3 Invertibility and Idempotency	
3.4 Schema Evolution and Conformance	
4. Primitive 3 — Deterministic Routing (DR)	9
4.1 Routing Table Schema	
4.2 Predicate Evaluation and Snapshot Semantics	
4.3 Dead-Letter Handling and Replay	
5. Primitive 4 — Seed Lifecycle (SL)	11
5.1 Seed Schema	
5.2 Lifecycle State Machine	
5.3 Compensation and Expiry	
6. Primitive 5 — Settlement Envelope (SE)	13
6.1 Settlement Envelope Schema	
6.2 Sealing and Verification	
6.3 Retention and Conformance	
7. Primitive 6 — DID Provenance (DP)	15
7.1 Supported DID Methods	
7.2 DID Document Requirements	
7.3 Provenance Chain and Key Lifecycle	

8. Primitive 7 — Universal Receipt (UR) 17

8.1 Receipt Schema

8.2 Locale, Verifiability, and Immutability

9. Primitive Interaction Map 19

10. Conformance Levels 20

11. Security Considerations 21

12. Versioning and Evolution Strategy 22

Appendix A: Glossary of Key Terms 23

Appendix B: Normative References 25

Executive Summary

Executive Summary

The Universal Resource and Identity Bus (URIB) is an open, interoperability-focused protocol architecture designed to address the critical deficits in modern distributed transaction systems: semantic drift across system boundaries, non-auditable routing decisions, identity ambiguity, and the absence of machine-verifiable settlement artifacts. This specification defines the URIB Canonical Architecture Stack — a coherent set of seven formally specified primitives that, together, provide a complete substrate for deterministic, auditable, and privacy-preserving distributed transactions across heterogeneous systems, jurisdictions, and trust domains.

The seven primitives are layered in order of dependency: the **Canonical Event Model (CEM)** establishes the universal data envelope; **Reversible Mapping (RM)** enables lossless protocol translation; **Deterministic Routing (DR)** ensures fully predictable and replay-safe message delivery; the **Seed Lifecycle (SL)** encodes atomic units of intent and their authorized state progressions; the **Settlement Envelope (SE)** produces tamper-evident, cryptographically sealed records of completed transactions; **DID Provenance (DP)** binds every actor and artifact to a

resolvable, standards-compliant decentralized identity; and the **Universal Receipt (UR)** bridges cryptographic certainty with human and regulatory comprehensibility. Each primitive is independently versionable yet normatively interoperable within the stack.

Implementers are assigned to one of three conformance tiers: **URIB-Core** (Primitives 1–3, suitable for event-driven integration platforms), **URIB-Standard** (Primitives 1–5, suitable for financial and transactional workflows), and **URIB-Full** (all seven primitives, required for regulated, identity-anchored settlement). The specification makes extensive use of RFC 2119 normative keywords (MUST, SHOULD, MAY) and is intended as a foundation for standards body adoption, enterprise integration, and cross-border interoperability agreements.

This document is structured to serve multiple audiences simultaneously: **protocol architects** will find formal schema definitions, invariant tables, and interaction maps; **distributed systems engineers** will find state machine specifications, algorithm descriptions, and conformance test vectors; **standards bodies** will find normative references to W3C DID Core, RFC 8785, RFC 7515, RFC 3339, and associated cryptographic standards; and **enterprise integration teams** will find conformance criteria, retention policy guidance, and jurisdiction-aware configuration parameters. Version 1.0.0 of this specification is submitted as a Draft for Publication and is open for public comment.

1. Introduction

1.1 Motivation

Contemporary distributed systems operate across boundaries of protocol, schema, identity, and jurisdiction. In the absence of a canonical architecture, each integration point becomes a bespoke negotiation — producing semantic drift, audit gaps, and non-interoperable settlement artifacts. The URIB stack addresses this by defining seven primitives that are individually minimal, collectively complete, and normatively composable.

1.2 Scope

This specification normatively defines the seven URIB primitives. It does not specify transport-layer protocols, physical network topology, or application-layer business logic beyond what is necessary to establish invariants. Implementers retain full latitude in their choice of infrastructure provided all normative constraints herein are satisfied.

1.3 Conventions

The key words **MUST**, **MUST NOT**, **REQUIRED**, **SHALL**, **SHALL NOT**, **SHOULD**, **SHOULD NOT**, **RECOMMENDED**, **MAY**, and **OPTIONAL** in this document are to be interpreted as described in RFC 2119. All JSON schema field names are rendered in `monospace`. Identifiers are case-sensitive unless explicitly stated otherwise.

1.4 Document Status

This document is a Draft for Publication. All schema definitions, state machines, and conformance criteria are normative. Sections marked with a  WARNING callout contain security-critical constraints that implementations **MUST NOT** omit.

Primitive 1 of 7

Canonical Event Model

CEM - Version 1.0.0 - urn:urib-primitive:cem:1.0.0

2. Primitive 1 — Canonical Event Model (CEM)

Purpose

The CEM is the foundational data envelope for all events flowing through the URIB stack. It establishes a single, unambiguous event shape that all participants produce and consume, eliminating semantic drift across system boundaries.

2.1 Normative Schema

Every CEM event **MUST** conform to the following schema. Fields marked **Required MUST** be present on every event. Fields marked **Conditional MUST** be present when the described condition is true. Fields marked **Optional MAY** be omitted.

Field Name	Type	Cardinality	Description
event_id	UUID v7	Required	Globally unique event identifier. MUST be UUID version 7 (time-ordered) to support monotonic event ordering without a central sequencer.
event_type	string	Required	Namespaced event type in the form namespace:domain:verb (e.g., urib:payments:initiated). All three segments MUST be present.
schema_version	semver string	Required	Semantic version of the event payload schema (e.g., 2.1.0). Consumers MUST reject events whose major version exceeds their supported range.
occurred_at	RFC 3339 timestamp	Required	Wall-clock time at which the event occurred, with nanosecond precision (e.g., 2026-07-02T09:52:00.000000000Z). MUST include UTC offset or Z.
source_uri	URI	Required	Fully qualified identifier of the producing system or service. MUST be dereferenceable or registered in the URIB service registry.
payload	object	Required	Typed, versioned event body. Structure is governed by the schema identified by event_type and schema_version.
correlation_id	UUID v7	Required	Shared across all events in a single logical operation or saga. Used for end-to-end tracing.
causation_id	UUID v7	Conditional	The event_id of the event that directly caused this event. MUST be present for all non-originating events in a causal chain.
lamport_clock	uint64	Required	Lamport timestamp for causal ordering. Producing systems MUST increment their local Lamport clock before emitting each event.

Field Name	Type	Cardinality	Description
<code>actor_did</code>	DID string	Required	W3C DID of the actor (human or system) that produced this event. MUST be resolvable at the time of emission.
<code>checksum</code>	string (hex)	Required	SHA-256 digest of the canonical JSON serialization of the event (per RFC 8785), computed over all fields except <code>checksum</code> itself.
<code>metadata</code>	object	Optional	Extensible key-value map for implementation-specific annotations. MUST NOT override or duplicate normative fields.

2.2 Event Classification Taxonomy

All CEM events **MUST** be assigned to exactly one of the following classifications, encoded in the first segment of `event_type`:

Classification	Prefix Segment	Semantics	Mutability
Command	<code>cmd</code>	An intent to perform an action. May be rejected. Addressed to a specific receiver.	Immutable once emitted
DomainEvent	<code>evt</code>	A fact that has occurred within a bounded context. Past tense. Cannot be rejected.	Immutable once emitted
IntegrationEvent	<code>int</code>	A DomainEvent published across context boundaries. Subject to Reversible Mapping transforms.	Immutable once emitted
SagaStep	<code>saga</code>	An event that advances or compensates a Seed Lifecycle. MUST carry a valid <code>seed_id</code> in metadata.	Immutable once emitted
AuditRecord	<code>audit</code>	A system-generated record of an administrative or security-relevant action. Elevated retention requirements apply.	Immutable once emitted

2.3 Canonical JSON Serialization and Ordering

2.3.1 Serialization

The canonical serialization of a CEM event **MUST** conform to RFC 8785 (JSON Canonicalization Scheme). The following rules apply in addition to RFC 8785:

- 1. Keys **MUST** be sorted lexicographically (Unicode code point order).
- 2. Floating-point numbers **MUST NOT** appear in any normative field; use integer or string representations.
- 3. The `checksum` field **MUST** be excluded from the input to the checksum computation and then appended to the serialized object prior to storage or transmission.
- 4. Unicode strings **MUST** be NFC-normalized before serialization.

2.3.2 Ordering Modes

Ordering Mode	Mechanism	Guarantee	When to Use
Causal Ordering	<code>causation_id chain</code> + <code>lamport_clock</code>	An event is delivered only after all events it causally depends on have been delivered.	Default mode for all Seed Lifecycle events.
Total Ordering	Monotonic sequence number assigned by the URIB broker	A strict global order exists across all events in a partition.	Required for AuditRecord events and Settlement Envelopes.

2.4 Immutability and Conformance

Invariant — CEM Immutability

Once emitted, a CEM event is **sealed** and content-addressed by its `checksum`. No field of an emitted event may be mutated. Any system that receives an event with a non-matching `checksum` **MUST** reject the event and record a routing failure.

Conformance Criteria — CEM

Producer: A compliant CEM producer MUST include all Required fields; MUST include all Conditional fields when conditions are met; MUST compute the checksum over the RFC 8785 canonical form; and MUST assign a UUID v7 to `event_id`.

Consumer: A compliant CEM consumer MUST validate the `checksum` before processing any event payload; MUST reject events with missing Required fields; and MUST surface a checksum mismatch as a processing error, not silently ignore it.

2.5 Interaction with Adjacent Primitives

Adjacent Primitive	Nature of Interaction
Primitive 2 — Reversible Mapping	CEM event payloads are translated between schemas using RM transform functions when crossing system boundaries. The CEM envelope is preserved; only the <code>payload</code> field is transformed.
Primitive 4 — Seed Lifecycle	SagaStep CEM events are the exclusive trigger mechanism for Seed state transitions. Each state transition MUST be traceable to exactly one CEM event.
Primitive 6 — DID Provenance	The <code>actor_did</code> field on every CEM event is resolved via DP. DP also supplies the linked-data proof attached to each event.

Primitive 2 of 7

Reversible Mapping

RM - Version 1.0.0 - urn:urib:primitive:rm:1.0.0

3. Primitive 2 — Reversible Mapping (RM)

Purpose

RM defines the bidirectional, lossless transformation layer between any two data representations within or across system boundaries. It enables zero-data-loss protocol translation and ensures that every transform is invertible under the same key set.

3.1 Mapping Definition Language

A URIB mapping is formally defined as an ordered triple:

```
Mapping := (source_schema, target_schema, transform_function) where:
source_schema      :: SchemaRef — content-addressed reference to the source
JSON Schema      target_schema      :: SchemaRef — content-addressed reference
to the target JSON Schema transform_function :: F : D(source_schema) →
D(target_schema)      where F is pure (no side effects)
and deterministic
```

Every registered mapping **MUST** also supply:

- An **inverse function** $F^{-1} : D(\text{target_schema}) \rightarrow D(\text{source_schema})$.
- At least one **round-trip test vector**: a sample input x such that $F^{-1}(F(x)) = x$ is verifiably true.

3.2 Mapping Registry

The URIB Mapping Registry is a versioned, content-addressed catalog of all registered transform triples. The registry schema is as follows:

Field	Type	Description
<code>mapping_id</code>	UUID v7	Unique identifier for this mapping entry.
<code>source_schema_ref</code>	content hash (SHA-256)	Content-addressed reference to the source schema document.
<code>target_schema_ref</code>	content hash (SHA-256)	Content-addressed reference to the target schema document.
<code>transform_ref</code>	content hash (SHA-256)	Content-addressed reference to the compiled transform function artifact.
<code>inverse_transform_ref</code>	content hash (SHA-256)	Content-addressed reference to the compiled inverse transform function artifact.
<code>test_vectors</code>	array of TestVector	One or more round-trip test vectors. MUST include at least one vector per registered mapping.
<code>mapping_version</code>	semver string	Semantic version of this mapping. Breaking changes MUST increment the major version.
<code>effective_from</code>	RFC 3339 timestamp	Timestamp from which this mapping version is valid.
<code>effective_until</code>	RFC 3339 timestamp or null	Timestamp until which this mapping is valid. Null indicates indefinitely active.
<code>registrar_did</code>	DID string	DID of the entity that registered this mapping. MUST sign the registry entry.

3.3 Conflict Resolution

When multiple mappings exist for the same `(source_schema, target_schema)` pair, the following precedence rules apply in descending order of priority:

5. **Explicit** — A mapping directly referenced by a routing rule's `transformation_ref` field takes absolute precedence.
6. **Inferred** — A mapping whose `source_schema_ref` and `target_schema_ref` exactly match the event's declared schemas.
7. **Default** — A system-registered fallback mapping, if one exists for the pair.

Invariant — Idempotency

Applying the same mapping to the same input **MUST** yield an identical output on every invocation. Transform functions **MUST NOT** read from or write to external state. Any transform that violates this invariant **MUST** be rejected at registration time.

3.4 Schema Evolution

Change Type	Classification	Required Action
Adding optional fields	Backward-compatible	SHOULD bump patch or minor version. Existing mappings remain valid.
Adding required fields	Breaking	MUST register a new mapping with incremented major version. Old version MUST remain active until <code>effective_until</code> elapses.
Renaming or removing fields	Breaking	MUST register a new mapping version. Migration period SHOULD be at least 90 days.
Changing field semantics	Breaking	MUST register a new mapping version with explicit documentation of the semantic change.

Conformance Criteria — RM

A compliant RM implementation MUST: (a) register an inverse function for every forward mapping; (b) validate all test vectors at registration time; (c) enforce idempotency by prohibiting stateful or non-deterministic transform functions; (d) apply conflict resolution precedence rules consistently; and (e) preserve the CEM event envelope when transforming only the `payload` field.

Primitive 3 of 7

Deterministic Routing

DR - Version 1.0.0 - urn:urib:primitive:dr:1.0.0

4. Primitive 3 — Deterministic Routing (DR)

Purpose

DR specifies how any event or message traverses the URIB topology in a fully predictable, replay-safe manner. Given the same input and routing table snapshot, DR MUST produce the same routing decision — making the system fully auditable and debuggable.

4.1 Routing Table Schema

Field	Type	Cardinality	Description
route_id	UUID v7	Required	Unique identifier for this routing rule.
match_predicate	JSONPath or CEL expression	Required	Boolean expression evaluated against the CEM event envelope. MUST be a pure, side-effect-free expression.
priority	integer (0–65535)	Required	Evaluation order. Higher values are evaluated first. Ties are broken by route_id lexicographic order.
target_endpoint_uri	URI	Required	The endpoint to which matching events are dispatched. MUST be registered in the URIB service registry.

Field	Type	Cardinality	Description
<code>transformation_ref</code>	RM mapping_id	Optional	Reference to an RM mapping to apply to the event payload before dispatch. If absent, payload is forwarded unmodified.
<code>effective_from</code>	RFC 3339 timestamp	Required	The route is active from this timestamp.
<code>effective_until</code>	RFC 3339 timestamp or null	Required	The route expires at this timestamp. Null indicates indefinitely active.
<code>snapshot_hash</code>	SHA-256 hex string	Required	Content hash of the routing table snapshot in which this rule participates. Computed over the canonical JSON of the full snapshot.

4.2 Predicate Evaluation and Snapshot Semantics

4.2.1 Evaluation Algorithm

8. Load the immutable routing table snapshot identified by `snapshot_hash`.
9. Sort all active routes in descending `priority` order.
10. For each route in order, evaluate `match_predicate` against the CEM event envelope.
11. The first route whose predicate evaluates to `true` is selected. Evaluation halts immediately (first-match-wins).
12. If no route matches, the event is unroutable; proceed to dead-letter handling (Section 4.3).
13. Record the selected `route_id` and `snapshot_hash` in the event's routing metadata before dispatch.

Invariant — Snapshot Immutability

Routing decisions are bound to a named, immutable routing table snapshot identified by its `snapshot_hash`. No mutable state is consulted at routing time. A routing table snapshot **MUST** NOT be modified after its `snapshot_hash` has been computed and published. Any modification constitutes a new snapshot with a new hash.

4.3 Dead-Letter Handling and Replay Guarantee

4.3.1 Dead-Letter Queue

An unroutable event — one for which no predicate matches in the active snapshot — **MUST** be written to the Dead-Letter Queue (DLQ) with:

- The full CEM event envelope, unmodified.
- A `routing_failure` metadata block containing: `snapshot_hash`, `failed_at` (RFC 3339 nanosecond), `failure_reason` (enum: `NO_MATCHING_ROUTE` | `ENDPOINT_UNREACHABLE` | `TRANSFORM_ERROR` | `PREDICATE_EVAL_ERROR`), and `dlq_entry_id` (UUID v7).

⚠ Warning — DLQ Integrity

Systems **MUST NOT** silently discard unroutable events. The DLQ **MUST** be durable and auditable. DLQ entries **MUST** be retained for a minimum of 30 days. DLQ entries **MUST** themselves be valid CEM events of type `audit:routing:dead_lettered`.

4.3.2 Replay Guarantee

Replaying an event with the same `snapshot_hash` **MUST** yield an identical routing path.

Implementations **MUST** store the `snapshot_hash` alongside every dispatched event record to enable future replay and audit.

Conformance Criteria — DR

A compliant DR implementation MUST: (a) evaluate predicates in strict priority-descending order; (b) bind every routing decision to an immutable snapshot; (c) write all unroutable events to a durable DLQ; (d) record `route_id` and `snapshot_hash` on every dispatched event; and (e) guarantee that identical inputs and snapshot produce identical routing outputs.

Primitive 4 of 7

Seed Lifecycle

SL - Version 1.0.0 - urn:urib:primitive:sl:1.0.0

5. Primitive 4 — Seed Lifecycle (SL)

Purpose

The Seed is the atomic unit of intent within URIB. A Seed encodes a desired state transition, carries its own authorization proof, and progresses through a strict lifecycle automaton from creation to terminal state. No state transition is permitted outside the defined lifecycle automaton.

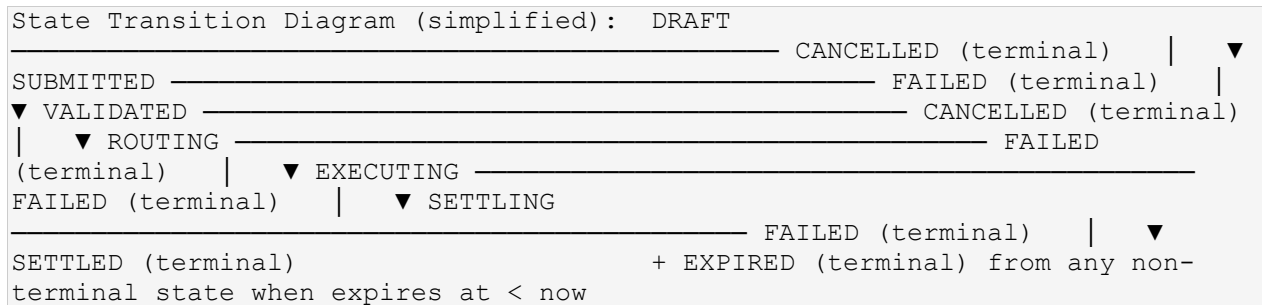
5.1 Seed Schema

Field	Type	Cardinality	Description
<code>seed_id</code>	UUID v7	Required	Globally unique seed identifier.
<code>seed_type</code>	namespaced string	Required	Categorizes the seed's intent (e.g., <code>urib:payment:transfer</code>).
<code>initiator_did</code>	DID string	Required	DID of the human or system that submitted the seed. Resolved via DP at submission time.
<code>authorization_proof</code>	JWT or UCAN	Required	Cryptographic proof that the initiator is authorized to request the desired state transition. MUST be verified before the SUBMITTED transition is accepted.
<code>desired_state</code>	object	Required	The target state the seed intends to bring about. Schema is governed by <code>seed_type</code> .
<code>current_lifecycle_state</code>	enum (see §5.2)	Required	The current state in the lifecycle automaton.
<code>created_at</code>	RFC 3339 nanosecond	Required	Timestamp of seed creation. Immutable after creation.
<code>updated_at</code>	RFC 3339 nanosecond	Required	Timestamp of the most recent state transition.
<code>expires_at</code>	RFC 3339 nanosecond or null	Required	If set, the seed MUST transition to EXPIRED if this timestamp passes without reaching SETTLED.
<code>parent_seed_id</code>	UUID v7 or null	Optional	Present on compensating (reversal) seeds. References the seed being compensated.
<code>child_seed_ids</code>	array of UUID v7	Optional	List of seeds spawned by this seed (sub-tasks or compensation seeds).
<code>metadata</code>	object	Optional	Extensible annotations. MUST NOT override normative fields.

5.2 Lifecycle State Machine

The following table defines all valid states, their terminal nature, and the allowed transitions. No transition not listed here is permitted.

State	Terminal?	Allowed Next States	Trigger Event Type	Actor
DRAFT	No	SUBMITTED, CANCELLED	saga:seed:submitted	Initiator DID
SUBMITTED	No	VALIDATED, FAILED	saga:seed:validated / saga:seed:validation_failed	URIB Validation Service
VALIDATED	No	ROUTING, CANCELLED	saga:seed:routing_started	URIB Router (DR)
ROUTING	No	EXECUTING, FAILED	saga:seed:execution_started	Target System DID
EXECUTING	No	SETTLING, FAILED	saga:seed:settlement_started	Settlement Channel
SETTLING	No	SETTLED, FAILED	saga:seed:settled / saga:seed:settlement_failed	Settlement Channel
SETTLED	Yes	—	None (terminal)	—
FAILED	Yes	—	None (terminal); triggers compensating seed	—
EXPIRED	Yes	—	None (terminal); triggered by expires_at clock	URIB Expiry Monitor
CANCELLED	Yes	—	None (terminal); triggers compensating seed if execution has begun	Initiator DID or authorized system



5.3 Compensation and Expiry

⚠ Warning — Compensation is Mandatory

FAILED and CANCELLED states on a Seed that has progressed beyond VALIDATED MUST trigger the creation of a compensating Seed. A compensating Seed is a first-class Seed with `parent_seed_id` set to the failed/cancelled Seed's `seed_id`. The compensating Seed follows the standard Saga compensation pattern and MUST reverse all effects of the original Seed's execution.

Invariant — Expiry is Irreversible

Seeds with `expires_at` in the past that have not reached a terminal state MUST be transitioned to EXPIRED by the URIB Expiry Monitor. No resurrection from EXPIRED is permitted under any circumstances. Implementations MUST NOT accept state transition events for EXPIRED seeds.

Conformance Criteria — SL

A compliant SL implementation MUST: (a) reject all state transitions not enumerated in the state machine table; (b) verify `authorization_proof` before accepting the SUBMITTED transition; (c) create a compensating Seed on any FAILED or CANCELLED transition beyond VALIDATED; (d) enforce expiry transitions without exception; and (e) emit a SagaStep CEM event for every state transition.

Primitive 5 of 7

Settlement Envelope

SE - Version 1.0.0 - urn:urib:primitive:se:1.0.0

6. Primitive 5 — Settlement Envelope (SE)

Purpose

The Settlement Envelope is the tamper-evident, cryptographically sealed record produced when a Seed reaches a terminal state. It is the canonical artifact of record for audit, reconciliation, and legal hold.

6.1 Settlement Envelope Schema

Field	Type	Cardinality	Description
envelope_id	UUID v7	Required	Globally unique identifier for this Settlement Envelope.
seed_id	UUID v7 (FK → Seed)	Required	Foreign key reference to the Seed that produced this envelope.
settled_at	RFC 3339 nanosecond	Required	Timestamp at which the terminal state was reached and the envelope was sealed.
settlement_status	enum	Required	One of: SETTLED, FAILED, CANCELLED, EXPIRED. Matches the terminal Seed state.
settlement_channel	URI	Required	Identifier of the channel or system that confirmed settlement.

Field	Type	Cardinality	Description
<code>final_state_hash</code>	SHA-256 hex string	Required	SHA-256 of the settled Seed record in canonical JSON (RFC 8785). Enables independent verification of the Seed's final state.
<code>event_log_hash</code>	SHA-256 hex string	Required	Merkle root of all CEM events in the Seed's lifecycle, ordered by <code>lamport_clock</code> . Enables verification of the complete event chain.
<code>issuer_did</code>	DID string	Required	DID of the entity that sealed and signed the envelope.
<code>issuer_signature</code>	detached JWS	Required	Ed25519 or P-256 detached JWS over the canonical JSON of the SE (excluding this field). Computed by the <code>issuer_did</code> 's current signing key.
<code>receipt_id</code>	UUID v7 (FK → Universal Receipt)	Required	Foreign key linking this SE to the corresponding Universal Receipt artifact.
<code>schema_version</code>	semver string	Required	Semantic version of the SE schema used to produce this envelope.

6.2 Sealing Process

The SE sealing process **MUST** proceed in the following order:

14. Compute `final_state_hash`: serialize the terminal Seed record to canonical JSON (RFC 8785) and compute SHA-256.
15. Collect all CEM events with `seed_id` in their `metadata`, sort by `lamport_clock` ascending.
16. Compute `event_log_hash`: construct a Merkle tree over the sorted event `checksum` values and record the root hash.
17. Populate all SE fields.
18. Serialize the SE (excluding `issuer_signature`) to canonical JSON.

19. Sign the canonical JSON using the `issuer_did`'s current `assertionMethod` key (Ed25519 or P-256). Produce a detached JWS.
20. Store `issuer_signature` in the SE record. The SE is now sealed.

Invariant — Envelope Immutability

Once sealed, no field of a Settlement Envelope MAY be mutated. Any system that receives an SE MUST verify the `issuer_signature` before relying on any field. Mutation of a sealed SE is a security violation and MUST be treated as tampering.

6.3 Verification Algorithm

Any party MUST be able to independently verify a Settlement Envelope by executing the following steps:

21. Resolve `issuer_did` using the W3C DID resolution algorithm. Retrieve the DID Document valid at `settled_at` (using `versionTime` DID URL parameter).
22. Locate the `assertionMethod` verification relationship in the resolved DID Document.
23. Serialize the SE (excluding `issuer_signature`) to canonical JSON (RFC 8785).
24. Verify the detached JWS in `issuer_signature` against the canonical JSON using the retrieved verification key.
25. Independently recompute the Merkle root from the event log and confirm it matches `event_log_hash`.
26. Recompute the `final_state_hash` from the terminal Seed record and confirm it matches.
27. If all checks pass, the SE is verified. If any check fails, the SE MUST be considered invalid.

6.4 Retention Policy

Parameter	Normative Minimum	Notes
Retention period	7 years from <code>settled_at</code>	Configurable upward by jurisdiction. Downward configuration MUST require explicit regulatory authority.
Geographic replication	Minimum 2 geographically distinct stores	Stores MUST be in different physical data centers and SHOULD be in different regulatory jurisdictions.
Integrity verification	Monthly re-verification of stored SEs	Failures MUST trigger an alert and re-replication within 24 hours.

Conformance Criteria — SE

A compliant SE implementation MUST: (a) seal the envelope using the exact sealing algorithm in §6.2; (b) produce a valid detached JWS using Ed25519 or P-256; (c) compute the Merkle root over all lifecycle events; (d) retain the SE for at minimum 7 years in at minimum 2 geographic locations; and (e) expose a verification endpoint that returns the SE in canonical JSON.

Primitive 6 of 7

DID Provenance

DP - Version 1.0.0 - urn:urib:primitive:dp:1.0.0

7. Primitive 6 — DID Provenance (DP)

Purpose

DID Provenance defines the binding of every actor, system, and artifact within URIB to a W3C Decentralized Identifier (DID), and the rules for resolving, verifying, and chaining identity claims across the lifecycle of a Seed. No action within URIB is accepted without a resolvable DID with a valid, non-revoked verification key.

7.1 Supported DID Methods

The URIB stack supports the following DID methods as of this specification version. Additional methods may be registered upon successful completion of URIB conformance testing.

DID Method	Identifier	Trust Model	Suitable For
did:web	did:web:<domain>	DNS + TLS (delegated trust)	Enterprise systems, B2B integrations
did:key	did:key:<multibase-pubkey>	Self-sovereign (no registry)	Ephemeral actors, testing, device identity
did:ion	did:ion:<sidetree-suffix>	Bitcoin-anchored (Sidetree)	High-assurance, cross-border transactions
did:ebssi	did:ebssi:<identifier>	EU Blockchain Service Infrastructure	EU regulatory contexts, public sector

7.2 DID Document Requirements

Every DID Document registered with URIB **MUST** meet the following requirements:

28. Include at least one `assertionMethod` verification relationship referencing a supported key type (Ed25519VerificationKey2020 or EcdsaSecp256k1VerificationKey2019 or JsnWebKey2020).
29. Include at least one `authentication` verification relationship.
30. Include at least one `service` endpoint of type `URIBEndpoint`, with a `serviceEndpoint` URI that is reachable and returns a URIB-compliant capability document.
31. **MUST NOT** use deactivated key material in any `assertionMethod` or `authentication` relationship.

7.3 Provenance Chain

The URIB provenance chain is a directed acyclic graph (DAG) of identity attestations recorded across the lifecycle of a Seed. Each node in the graph is a CEM event carrying:

- The actor's `actor_did`.
- A linked-data proof (JSON-LD Proof or JWS) produced by the actor's current `assertionMethod` key.
- A reference to the prior node via `causation_id`.

The complete provenance graph — from Seed initiator through each routing hop to final settlement — **MUST** be traversable and verifiable by any auditor holding a copy of the event log.

7.4 Key Rotation and Revocation

Operation	Effect on Past Events	Effect on Future Operations	Normative Rule
Key Rotation	None — past events are verified against the DID Document version at the time of signing (using <code>versionTime</code> or <code>versionId</code> DID URL parameter).	New operations MUST use the new key. Old key MAY remain in the DID Document for a transition period.	Implementations MUST support historical DID Document resolution for audit purposes.
Key Revocation	None — already-settled envelopes MUST NOT be retroactively invalidated.	Revoked keys MUST NOT be used for new operations. Any attempt MUST be rejected.	Revocation MUST be propagated to all URIB participants within 1 hour of publication.

⚠ Warning — DID Resolution Trust

URIB participants **MUST NOT** cache DID Documents beyond their stated TTL. Stale DID resolution results can lead to acceptance of revoked keys. Implementations **MUST** use a DID Universal Resolver or equivalent that supports all four registered DID methods and their historical document retrieval capabilities.

Conformance Criteria — DP

A compliant DP implementation **MUST**: (a) refuse any operation from an actor whose DID cannot be resolved; (b) refuse any operation using a revoked verification key; (c) record the actor DID and linked-data proof on every CEM event; (d) support historical DID Document retrieval for audit verification; and (e) propagate key revocation within 1 hour.

Primitive 7 of 7

Universal Receipt

UR - Version 1.0.0 - urn:urib:primitive:ur:1.0.0

8. Primitive 7 — Universal Receipt (UR)

Purpose

The Universal Receipt is the human-readable, machine-verifiable summary artifact issued at the conclusion of every Seed lifecycle. It bridges the gap between the cryptographic certainty of the Settlement Envelope and the comprehensibility required by end-users, regulators, and auditors.

8.1 Receipt Schema

Field	Type	Cardinality	Description
<code>receipt_id</code>	UUID v7	Required	Globally unique identifier for this Universal Receipt.
<code>envelope_id</code>	UUID v7 (FK → SE)	Required	Foreign key reference to the Settlement Envelope that backs this receipt.
<code>issued_at</code>	RFC 3339 nanosecond	Required	Timestamp at which the receipt was issued. MUST be at or after the SE's <code>settled_at</code> .
<code>issuer_did</code>	DID string	Required	DID of the entity issuing the receipt. MUST match or be delegated by the SE's <code>issuer_did</code> .
<code>recipient_did</code>	DID string	Required	DID of the primary recipient of the receipt (typically the Seed initiator or their designated agent).
<code>receipt_type</code>	enum	Required	One of: <code>TRANSACTION</code> , <code>REFUND</code> , <code>CANCELLATION</code> , <code>AUDIT</code> .
<code>summary</code>	string	Required	Structured, locale-aware plain-language description of the transaction. MUST be rendered in the specified <code>locale</code> .
<code>line_items</code>	array of <code>LineItem</code>	Required	Each item: { <code>label</code> , <code>value</code> , <code>currency_or_unit</code> , <code>category</code> }. Values MUST be locale-neutral numbers; formatting is applied at render time.
<code>totals</code>	object	Required	Aggregated totals by category and overall. MUST be computable from <code>line_items</code> .
<code>currency_code</code>	string	Required	ISO 4217 currency code or a registered URIB custom token identifier.
<code>verification_uri</code>	HTTPS URI	Required	Endpoint that, when called, returns the full SE in canonical JSON. MUST return HTTP 200 with content-type <code>application/json</code> .
<code>qr_payload</code>	compact signed JWT	Required	Compact JWT encoding <code>envelope_id</code> and <code>verification_uri</code> , signed by

Field	Type	Cardinality	Description
			<code>issuer_did</code> . Suitable for QR code embedding.
<code>locale</code>	BCP 47 language tag	Required	Locale for rendering <code>summary</code> and <code>line_items</code> labels (e.g., <code>en-US</code> , <code>de-DE</code> , <code>ja-JP</code>).
<code>schema_version</code>	semver string	Required	Semantic version of the UR schema.
<code>supersedes_receipt_id</code>	UUID v7 or null	Conditional	MUST be present if this receipt supersedes a prior receipt. References the <code>receipt_id</code> of the superseded receipt.

8.2 Locale, Verifiability, and Immutability

8.2.1 Internationalization

The `summary` and `line_items` labels **MUST** be rendered in the language specified by `locale`. Underlying numerical values **MUST** be stored as locale-neutral data. Implementations **SHOULD** support all IETF BCP 47 language tags and **MUST** support at minimum: `en-US`, `en-GB`, `es-ES`, `fr-FR`, `de-DE`, `zh-CN`, `ja-JP`.

8.2.2 Verifiability

Any party scanning the QR code or calling `verification_uri` **MUST** receive a 200 response with the full SE in canonical JSON, enabling end-to-end verification without trusting the receipt's human-readable contents. The verification endpoint **MUST NOT** require authentication for public receipts.

Invariant — Receipt Immutability

Once issued, a Universal Receipt is immutable. No field may be modified. Corrections **MUST** result in a new UR with `supersedes_receipt_id` set to the original receipt's `receipt_id`. The original receipt **MUST** remain accessible and **MUST** be marked as superseded in the URIB receipt registry.

Conformance Criteria — UR

A compliant UR implementation MUST: (a) issue a receipt for every terminal Seed state; (b) link the receipt bidirectionally to the SE via `envelope_id` and `receipt_id`; (c) render locale-specific content from locale-neutral stored data; (d) maintain a reachable `verification_uri` for the retention lifetime of the linked SE; and (e) sign the `qr_payload` JWT using the `issuer_did`'s current key.

9. Primitive Interaction Map

9.1 Overview

The seven URIB primitives do not operate in isolation. They form a tightly integrated stack in which every primitive depends on or communicates with one or more others. Understanding these interactions is essential for correct implementation sequencing and conformance testing. The primary dependency flow is: CEM → RM → DR → SL → SE → DP (cross-cutting) → UR. DID Provenance (DP) is the sole cross-cutting primitive: it interacts normatively with all other six primitives and MUST be implemented before any other primitive can be declared conformant.

9.2 Pairwise Interaction Matrix

Primitive Pair	Interacts?	Nature of Interaction	Direction
CEM ↔ RM	Yes	RM transforms the CEM event <code>payload</code> field between schemas at integration boundaries. The CEM envelope is preserved intact.	CEM provides the envelope; RM modifies the payload.
CEM ↔ DR	Yes	DR evaluates predicates against the CEM event envelope fields. The routing decision and <code>snapshot_hash</code> are written to the CEM event's routing metadata.	CEM is the input to DR; DR annotates CEM with routing metadata.
CEM ↔ SL	Yes	SagaStep CEM events are the exclusive mechanism for advancing Seed state transitions. Every Seed state change MUST be traceable to exactly one CEM event.	CEM drives SL; SL produces CEM events as side effects of transitions.
CEM ↔ SE	Yes	The SE's <code>event_log_hash</code> is the Merkle root of all CEM events in a Seed's lifecycle. CEM event checksums are the leaf nodes of this Merkle tree.	CEM provides the event log; SE seals it cryptographically.
CEM ↔ DP	Yes	Every CEM event carries <code>actor_did</code> resolved via DP and a linked-data proof produced by the actor's DP-registered key.	DP supplies identity material to CEM.
CEM ↔ UR	Indirect	No direct schema interaction. CEM events flow through SL and SE before UR is produced. UR references the SE, which references the CEM event log.	Indirect, via SE.
RM ↔ DR	Yes	DR routing rules reference RM mappings via <code>transformation_ref</code> . The selected RM mapping is applied to the CEM event payload before dispatch to the target endpoint.	DR invokes RM at routing time.
RM ↔ SL	Indirect	RM may transform identity representations (e.g., DID formats) referenced by SL fields during cross-boundary integration. Not a direct schema dependency.	Indirect, via CEM payload translation.
RM ↔ DP	Yes	RM mappings may translate between identity representation formats (e.g., different DID method syntaxes). DP provides validation of DID correctness after transformation.	RM transforms identity representations; DP validates them.

Primitive Pair	Interacts?	Nature of Interaction	Direction
DR ↔ SL	Yes	Seeds (as typed payloads in CEM events) are routed by DR to their target settlement channel. The DR decision triggers the ROUTING state transition in SL.	DR dispatches Seeds; the dispatch event triggers SL transition.
DR ↔ SE	Yes	DR attaches routing metadata (<code>route_id</code> , <code>snapshot_hash</code>) to CEM events that form part of the SE's event log. This metadata is captured in the Merkle root.	DR metadata flows into SE's <code>event_log_hash</code> .
SL ↔ SE	Yes	A terminal Seed state (SETTLED, FAILED, CANCELLED, EXPIRED) triggers production of exactly one SE. The SE includes a FK to the Seed via <code>seed_id</code> .	SL terminal state is the precondition for SE creation.
SL ↔ DP	Yes	The <code>initiator_did</code> on a Seed is resolved via DP at SUBMITTED transition time. The <code>authorization_proof</code> is verified against the initiator's DP-registered key.	DP validates SL's initiator identity and authorization proof.
SE ↔ UR	Yes	The UR references the SE via <code>envelope_id</code> . The SE back-references the UR via <code>receipt_id</code> . This bidirectional FK pair is the primary link between machine-verifiable settlement and human-readable receipt.	Bidirectional FK reference.
SE ↔ DP	Yes	The SE is sealed using the <code>issuer_did</code> 's DP-registered signing key. Verification of the SE requires resolving <code>issuer_did</code> and retrieving the DID Document valid at <code>settled_at</code> .	DP supplies signing material for SE sealing and provides resolution for verification.
UR ↔ DP	Yes	The UR's <code>issuer_did</code> and <code>recipient_did</code> are URIB DIDs registered in DP. The <code>qr_payload</code> JWT is signed using the <code>issuer_did</code> 's DP-registered key.	DP provides identity anchoring for UR's issuer and recipient.

10. Conformance Levels

URIB defines three conformance tiers. Each tier is a strict superset of the prior tier. Partial conformance within a tier is not recognized; an implementation either satisfies all criteria for a tier or it does not.

Conformance Level	Required Primitives	Target Use Case	Certification Path
URIB-Core	Primitives 1 (CEM), 2 (RM), 3 (DR)	Event-driven integration platforms, message brokers, API gateways requiring deterministic routing and lossless schema translation.	Pass URIB-Core conformance test suite (CTS-CORE-001). Self-certifiable with published test vectors.
URIB-Standard	Primitives 1–5 (adds SL and SE)	Financial transaction systems, supply chain platforms, regulated data workflows requiring atomic intent tracking and cryptographic settlement artifacts.	Pass URIB-Standard CTS (CTS-STD-001). Requires third-party audit for regulated financial contexts.
URIB-Full	All seven primitives (adds DP and UR)	Cross-border payment rails, identity-anchored data exchanges, legal-hold systems, and any context requiring full end-to-end auditability, decentralized identity, and human-verifiable receipts.	Pass URIB-Full CTS (CTS-FULL-001). Requires certification by an accredited URIB Conformance Authority.

Primitive	URIB-Core	URIB-Standard	URIB-Full
1 — Canonical Event Model (CEM)	Required	Required	Required
2 — Reversible Mapping (RM)	Required	Required	Required
3 — Deterministic Routing (DR)	Required	Required	Required
4 — Seed Lifecycle (SL)	Not Required	Required	Required
5 — Settlement Envelope (SE)	Not Required	Required	Required
6 — DID Provenance (DP)	Not Required	Not Required	Required

Primitive	URIB-Core	URIB-Standard	URIB-Full
7 — Universal Receipt (UR)	Not Required	Not Required	Required

Note — Partial Implementation

Organizations implementing only a subset of the primitives within a tier MAY do so for internal use, but MUST NOT represent themselves as URIB-conformant at that tier level. Interoperability guarantees apply only between fully conformant implementations at the same or compatible tier.

11. Security Considerations

11.1 Key Rotation

All signing keys used within the URIB stack **MUST** be rotated at intervals not exceeding 12 months. Implementations **SHOULD** support automated key rotation with zero-downtime key transitions. During rotation, both the old and new keys **MUST** be simultaneously active in the DID Document for a transition period of no less than 24 hours.

11.2 Replay Attack Prevention

The CEM's UUID v7 `event_id` and `lamport_clock` together provide replay detection capability. Consumers **MUST** maintain an idempotency store keyed on `event_id` with a minimum retention of 30 days. Any event whose `event_id` has already been processed **MUST** be silently acknowledged but not reprocessed. Implementations **MUST NOT** re-process duplicate events even if they are delivered via a different transport or routing path.

⚠ Warning — Replay via DLQ

Intentional replay from the DLQ for recovery purposes **MUST** be distinguished from malicious replay by including the original `event_id` in the idempotency store check. DLQ-sourced replays **MUST** be explicitly authorized by an operator with a valid DID and logged as an AuditRecord CEM event.

11.3 DID Resolution Trust Anchors

The security of the entire URIB stack depends on the integrity of DID resolution. Implementations **MUST**:

32. Configure DID Universal Resolver instances with pinned trust anchors appropriate to each DID method.
33. Validate TLS certificates on `did:web` resolution endpoints against a trusted CA store.
34. Verify Sidetree proofs for `did:ion` against a Bitcoin node operated by or trusted by the implementer.
35. Never accept DID resolution results from untrusted intermediaries or caching proxies beyond their stated TTL.

11.4 Merkle Proof Completeness

The Settlement Envelope's `event_log_hash` is only as secure as the completeness of the event log it summarizes. Implementations **MUST** ensure that the event log submitted for Merkle root computation is complete — containing all CEM events in the Seed's lifecycle without gaps. Gaps in the event log **MUST** cause SE sealing to fail. Auditors performing SE verification **MUST** independently source the event log from a durable store and compare the recomputed Merkle root against `event_log_hash`.

11.5 Canonical JSON Side-Channel Risks

RFC 8785 canonicalization is computationally deterministic; however, implementations **SHOULD** be aware that timing side-channels during canonicalization may leak structural information about the event payload. Production implementations performing canonicalization in security-sensitive contexts **SHOULD** use constant-time string sorting routines where available and **SHOULD** avoid logging intermediate canonicalization states.

11.6 Authorization Proof Validation

The Seed's `authorization_proof` (JWT or UCAN) **MUST** be validated in full before the SUBMITTED transition is accepted. Validation **MUST** include: signature verification against the issuer's current DID key, expiry check, audience claim verification, and capability scope verification (for UCAN proofs). Expired or malformed authorization proofs **MUST** cause immediate rejection with a structured error CEM event.

12. Versioning and Evolution Strategy

12.1 Per-Primitive Semantic Versioning

Each of the seven URIB primitives carries an independent semantic version in the format `MAJOR.MINOR.PATCH`. Version tags are embedded in the primitive's URN identifier (e.g., `urn:urib:primitive:cem:1.0.0`). Primitives evolve independently; a breaking change in one primitive does not require a version bump in any other.

12.2 Compatibility Matrix (v1.0.0 Baseline)

Primitive	Current Version	Backward-Compatible From	Breaking Change Policy
CEM	1.0.0	1.0.0 (baseline)	Major version bump; 180-day migration window
RM	1.0.0	1.0.0 (baseline)	Major version bump; 90-day migration window
DR	1.0.0	1.0.0 (baseline)	Major version bump; 90-day migration window
SL	1.0.0	1.0.0 (baseline)	Major version bump; 180-day migration window; lifecycle state additions treated as minor
SE	1.0.0	1.0.0 (baseline)	Major version bump; 365-day migration window (due to long-lived archived envelopes)
DP	1.0.0	1.0.0 (baseline)	New DID method additions treated as minor; breaking resolver changes require major bump
UR	1.0.0	1.0.0 (baseline)	Major version bump; 90-day migration window

12.3 Deprecation Policy

36. **Announcement:** Deprecation of a primitive version or feature **MUST** be announced on the URIB Standards mailing list and in the official change log with at minimum 180 days notice before the effective date.
37. **Dual Support:** During the deprecation window, implementations **MUST** support both the deprecated and the replacement version.
38. **End of Life:** After the deprecation window closes, implementations **MAY** drop support for the deprecated version. Implementations that do so **MUST** surface a clear error to callers presenting the deprecated version.
39. **Security Exceptions:** If a security vulnerability is discovered in a primitive version, an emergency deprecation notice **MAY** be issued with a window as short as 7 days. All URIB-certified implementations **MUST** subscribe to the URIB Security Advisory mailing list.

Appendix A

Glossary of Key Terms

Appendix A: Glossary of Key Terms

Authorization Proof — A cryptographic token, either a JSON Web Token (JWT) or a User-Controlled Authorization Network (UCAN) capability token, that demonstrates the bearer's authorization to perform a specific action within the URIB stack. Required on every Seed at submission time.

Canonical Event Model (CEM) — Primitive 1 of the URIB stack. The universal, normatively defined data envelope for all events flowing through the stack. Establishes a single unambiguous event shape including identity, causation, ordering, and integrity fields.

Canonical JSON — A deterministic, byte-for-byte identical serialization of a JSON object produced by following RFC 8785 (JSON Canonicalization Scheme). Used as the input to all cryptographic operations in the URIB stack.

Causation ID — The `event_id` of the CEM event that directly caused the current event. Used to construct the causal provenance chain of a Seed's lifecycle.

Conformance Authority — An accredited body authorized to issue URIB-Full conformance certifications following successful completion of the CTS-FULL-001 test suite.

Correlation ID — A UUID v7 shared across all CEM events belonging to a single logical operation or saga. Used for end-to-end distributed tracing.

Dead-Letter Queue (DLQ) — A durable, auditable store to which unroutable CEM events are written by the Deterministic Routing primitive when no matching routing rule can be found.

Decentralized Identifier (DID) — A W3C-standardized globally unique persistent identifier that enables verifiable, decentralized digital identity. DIDs are resolved to DID Documents containing verification keys and service endpoints.

Deterministic Routing (DR) — Primitive 3 of the URIB stack. The component that routes CEM events through the URIB topology in a fully predictable, snapshot-bound, and replay-safe manner.

DID Document — The JSON-LD document retrieved by resolving a DID. Contains verification keys (as verification relationships), service endpoints, and other DID subject metadata.

DID Provenance (DP) — Primitive 6 of the URIB stack. The cross-cutting identity fabric that binds every actor, system, and artifact to a resolvable DID and governs key rotation and revocation rules.

Ed25519 — An elliptic curve digital signature algorithm (EdDSA) using Curve25519. The preferred signing algorithm for URIB Settlement Envelope signatures and JWT/JWS operations due to its performance and security characteristics.

Event Log Hash — The Merkle root computed over the SHA-256 checksums of all CEM events in a Seed's lifecycle, ordered by Lamport clock. Stored in the Settlement Envelope's `event_log_hash` field.

Idempotency Store — A persistent data store, keyed on CEM `event_id`, maintained by consumers to detect and suppress duplicate event deliveries.

JSON Web Signature (JWS) — A standard (RFC 7515) for representing content secured with digital signatures using JSON-based data structures. Used for Settlement Envelope sealing and Universal Receipt QR payloads.

JSON Web Token (JWT) — A compact, URL-safe token standard (RFC 7519) for representing claims securely between parties. Used as the authorization proof format on URIB Seeds and as the QR payload in Universal Receipts.

Lamport Clock — A logical clock mechanism that assigns a monotonically increasing integer to each event, enabling causal ordering without reliance on synchronized wall clocks. Every CEM event carries a `lamport_clock` field.

Merkle Root — The single hash value at the top of a Merkle tree, which cryptographically summarizes all leaf hashes and allows any individual leaf to be verified with a logarithmic number of proof steps.

Reversible Mapping (RM) — Primitive 2 of the URIB stack. The bidirectional, lossless transformation layer that enables zero-data-loss schema and protocol translation between any two registered data representations.

Saga Pattern — A distributed transaction pattern in which a multi-step process is decomposed into a sequence of local transactions, each publishing an event. Failure triggers a series of compensating transactions in reverse order.

Seed — The atomic unit of intent within the URIB stack. A Seed encodes a desired state transition, carries its own authorization proof, and progresses through the Seed Lifecycle automaton from DRAFT to a terminal state.

Seed Lifecycle (SL) — Primitive 4 of the URIB stack. The normatively defined state machine governing all valid states and transitions of a URIB Seed, from DRAFT through to terminal states (SETTLED, FAILED, EXPIRED, CANCELLED).

Settlement Envelope (SE) — Primitive 5 of the URIB stack. The tamper-evident, cryptographically sealed record produced when a Seed reaches a terminal state. The canonical artifact for audit, reconciliation, and legal hold.

Snapshot Hash — The SHA-256 content hash of an immutable routing table snapshot. All DR routing decisions are bound to a specific snapshot hash, enabling full replay fidelity.

UCAN (User-Controlled Authorization Network) — A capability-based authorization token format that allows fine-grained, delegatable authorization without a central authority. Supported as an alternative to JWT for URIB Seed authorization proofs.

Universal Receipt (UR) — Primitive 7 of the URIB stack. The human-readable, machine-verifiable summary artifact issued at the conclusion of every Seed lifecycle, bridging cryptographic certainty with regulatory and end-user comprehensibility.

URIB (Universal Resource and Identity Bus) — The open protocol architecture defined by this specification. URIB provides a canonical seven-primitive stack for deterministic, auditable, and privacy-preserving distributed transactions across heterogeneous systems, jurisdictions, and trust domains.

URN (Uniform Resource Name) — A persistent, location-independent identifier. URIB uses URNs to identify primitive versions (e.g., `urn:urib:primitive:cem:1.0.0`).

UUID v7 — A universally unique identifier variant that encodes a millisecond-precision timestamp in the most significant bits, enabling monotonic, time-ordered generation without a central sequencer. Required for all `event_id`, `seed_id`, and `envelope_id` fields.

Appendix B

Normative References

Appendix B: Normative References

The following documents are normatively referenced by this specification. Implementations claiming URIB conformance MUST comply with the referenced requirements as cited herein.

Reference	Document	Used By
[W3C-DID]	W3C Decentralized Identifiers (DIDs) v1.0. W3C Recommendation, 19 July 2022. https://www.w3.org/TR/did-core/	DP, SE, SL, UR
[RFC8785]	Anders Rundgren, Bret Jordan, Samuel Erdtman. "JSON Canonicalization Scheme (JCS)." RFC 8785. IETF, June 2020.	CEM, SE, RM
[RFC3339]	Graham Klyne, Chris Newman. "Date and Time on the Internet: Timestamps." RFC 3339. IETF, July 2002.	CEM, SL, SE, UR
[RFC7515]	Michael Jones, John Bradley, Nat Sakimura. "JSON Web Signature (JWS)." RFC 7515. IETF, May 2015.	SE, DP, UR
[RFC7519]	Michael Jones, John Bradley, Nat Sakimura. "JSON Web Token (JWT)." RFC 7519. IETF, May 2015.	SL, UR
[RFC2119]	Scott Bradner. "Key words for use in RFCs to Indicate Requirement Levels." RFC 2119. IETF, March 1997.	All primitives
[Ed25519]	Daniel J. Bernstein et al. "High-speed high-security signatures." RFC 8032 (EdDSA). IETF, January 2017.	SE, DP
[UCAN]	Brooklyn Zelenka, Fission Codes. "UCAN Specification v0.10.0." https://github.com/ucan-wg/spec	SL
[SHA-256]	NIST. "Secure Hash Standard (SHS)." FIPS PUB 180-4. National Institute of Standards and Technology, August 2015.	CEM, SE, RM, DR
[BCP47]	A. Phillips, M. Davis. "Tags for Identifying Languages." BCP 47. IETF, September 2009.	UR

Reference	Document	Used By
[ISO4217]	International Organization for Standardization. "Codes for the representation of currencies." ISO 4217:2015.	UR
[UUID7]	K. Davis, B. Peabody, P. Leach. "New UUID Formats." RFC 9562. IETF, May 2024.	CEM, SL, SE, UR

URIB Canonical Architecture Stack — Specification of the Seven Core Primitives

Document ID: URIB-SPEC-001 · Version 1.0.0 · Status: Draft for Publication · Date: July 2, 2026

Classification: Public · © Leon Calvin Long II · All normative content subject to URIB open specification license.